CHAPTER 4: CODING AND OUTPUT

4.1 Coding:

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <vector>
#include <iomanip>
#include <limits>
#include <cstdio>
#include <cctype>
#include <algorithm>
using namespace std;
namespace SMS {
const string BACK = "back";
const string EXIT = "exit";
void cls() {
#ifdef _WIN32
   system("cls");
#else
   system("clear");
#endif
}
void pauseScreen() {
   cout << "\nPress Enter to continue...";
   cin.ignore(numeric_limits<streamsize>::max(), '\n');
}
string lowerCopy(string s) {
   transform(s.begin(), s.end(), s.begin(),
         [](unsigned char c) { return static_cast<char>(tolower(c)); });
```

```
    return s;
}
bool isBack(const string& s) {
    return lowerCopy(s) == BACK;
}
bool isExit(const string& s) {
    return lowerCopy(s) == EXIT;
}
bool inputText(const string& prompt, string& value, bool allowEmpty = false) {
    while (true) {
        cout << prompt;
        getline(cin >> ws, value);
        if (isBack(value)) return false;
        if (isExit(value)) return false;
        if (!allowEmpty && value.empty()) {
            cout << "Invalid input\n";
            continue;
        }
        return true;
    }
}
bool inputInt(const string& prompt, int& value, int minValue = numeric_limits<int>::min(),
              int maxValue = numeric_limits<int>::max()) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        stringstream ss(input);
        char extra;
        if (!(ss >> value) || (ss >> extra) || value < minValue || value > maxValue) {
```

```
            cout << "Invalid option\n";
            continue;
        }
        return true;
    }
}
bool inputDouble(const string& prompt, double& value, double minValue = 0) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        stringstream ss(input);
        char extra;
        if (!(ss >> value) || (ss >> extra) || value < minValue) {
            cout << "Invalid option\n";
            continue;
        }
        return true;
    }
}
bool inputChar(const string& prompt, char& value) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        if (input.size() != 1) {
            cout << "Invalid option\n";
            continue;
        }
```

```
        value = input[0];
        return true;
    }
}
void invalidChoice() {
    cout << "\nInvalid option\n";
    pauseScreen();
}
bool menuChoice(const string& prompt, int& choice, int minChoice, int maxChoice) {
    while (true) {
        cout << prompt;
        string input;
        getline(cin >> ws, input);
        if (isBack(input) || isExit(input)) return false;
        stringstream ss(input);
        char extra;
        if (!(ss >> choice) || (ss >> extra) || choice < minChoice || choice > maxChoice) {
            invalidChoice();
            cls();
            return true;
        }
        return true;
    }
}
class FileManager {
public:
    static string nextID(const string& fileName, const string& prefix) {
        ifstream file(fileName);
        string line;
        int maxNumber = 0;
        while (getline(file, line)) {
```

```
        size_t pos = line.find('|');
        string id = (pos == string::npos) ? line : line.substr(0, pos);
        if (id.rfind(prefix, 0) == 0 && id.size() > prefix.size()) {
            string number = id.substr(prefix.size());
            bool numeric = !number.empty() &&
                all_of(number.begin(), number.end(),
                    [](unsigned char c) { return isdigit(c); });
            if (numeric) {
                try {
                    maxNumber = max(maxNumber, stoi(number));
                } catch (...) {}
            }
        }
    }
    ostringstream out;
    out << prefix << setw(3) << setfill('0') << (maxNumber + 1);
    return out.str();
}
static bool idExists(const string& fileName, const string& id) {
    ifstream file(fileName);
    string line;
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == id) return true;
    }
    return false;
}
static bool findByID(const string& fileName, const string& id, string& lineOut) {
    ifstream file(fileName);
    string line;
```

```
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == id) {
            lineOut = line;
            return true;
        }
    }
    return false;
}
static vector<string> readAll(const string& fileName) {
    vector<string> lines;
    ifstream file(fileName);
    string line;
    while (getline(file, line)) {
        if (!line.empty()) lines.push_back(line);
    }
    return lines;
}
static bool append(const string& fileName, const string& data) {
    ofstream file(fileName, ios::app);
    if (!file.is_open()) return false;
    file << data << '\n';
    return true;
}
static bool removeByID(const string& fileName, const string& id) {
    ifstream file(fileName);
    ofstream temp("temp_sms.txt");
    if (!file.is_open() || !temp.is_open()) return false;
    bool removed = false;
    string line;
```

```
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == id) {
            removed = true;
        } else {
            temp << line << '\n';
        }
    }
    file.close();
    temp.close();
    remove(fileName.c_str());
    rename("temp_sms.txt", fileName.c_str());
    return removed;
}
static bool removeSchoolByID(const string& schoolID) {
    return removeByID("schools.txt", schoolID);
}
static bool updateLine(const string& fileName, const string& matchID,
                       const string& newLine) {
    ifstream file(fileName);
    ofstream temp("temp_sms.txt");
    if (!file.is_open() || !temp.is_open()) return false;
    bool updated = false;
    string line;
    while (getline(file, line)) {
        size_t pos = line.find('|');
        string stored = (pos == string::npos) ? line : line.substr(0, pos);
        if (stored == matchID) {
            temp << newLine << '\n';
            updated = true;
```

```
        } else {
            temp << line << '\n';
        }
    }
    file.close();
    temp.close();
    remove(fileName.c_str());
    rename("temp_sms.txt", fileName.c_str());
    return updated;
  }
  static bool saveSchool(const string& data) { return append("schools.txt", data); }
  static bool savePrincipal(const string& data) { return append("principals.txt", data); }
  static bool saveTeacher(const string& data) { return append("teachers.txt", data); }
  static bool saveStudent(const string& data) { return append("students.txt", data); }
  static bool saveParent(const string& data) { return append("parents.txt", data); }
  static bool saveAttendance(const string& data) { return append("attendance.txt", data); }
  static bool saveResult(const string& data) { return append("results.txt", data); }
  static bool saveFees(const string& data) { return append("fees.txt", data); }
  static bool saveNotification(const string& data) { return append("notifications.txt", data);
}
  static bool saveComplaint(const string& data) { return append("complaints.txt", data); }
  static bool saveLeaveRequest(const string& data) { return append("leave_requests.txt",
data); }
  static bool saveRemovalRequest(const string& data) { return
append("removal_requests.txt", data); }
  static void load(const string& fileName) {
    vector<string> lines = readAll(fileName);
    if (lines.empty()) {
        cout << "No records found.\n";
        return;
    }
    for (const string& line : lines) cout << line << '\n';
```

```
}
static void loadSchools() { load("schools.txt"); }
static void loadPrincipals() { load("principals.txt"); }
static void loadTeachers() { load("teachers.txt"); }
static void loadStudents() { load("students.txt"); }
static void loadAttendance() { load("attendance.txt"); }
static void loadResults() { load("results.txt"); }
static void loadFees() { load("fees.txt"); }
static void loadNotifications() { load("notifications.txt"); }
static void loadComplaints() { load("complaints.txt"); }
static void loadLeaveRequests() { load("leave_requests.txt"); }
static void loadRemovalRequests() { load("removal_requests.txt"); }
static bool verifyLogin(const string& fileName, const string& id, const string& password) {
    string line;
    if (!findByID(fileName, id, line)) return false;
    vector<string> fields = split(line);
    return fields.size() >= 2 && fields[0] == id && fields[1] == password;
}
static bool verifyStudentLogin(const string& id, const string& password) {
    return verifyLogin("students.txt", id, password);
}
static bool verifyPrincipalLogin(const string& id, const string& password) {
    return verifyLogin("principals.txt", id, password);
}
static bool verifyTeacherLogin(const string& id, const string& password) {
    return verifyLogin("teachers.txt", id, password);
}
static bool verifyParentLogin(const string& id, const string& password) {
    return verifyLogin("parents.txt", id, password);
}
static vector<string> split(const string& line) {
```

```
    vector<string> fields;
    string field;
    stringstream ss(line);
    while (getline(ss, field, '|')) fields.push_back(field);
    return fields;
}
static string getField(const string& fileName, const string& id, size_t index) {
    string line;
    if (!findByID(fileName, id, line)) return "";
    vector<string> fields = split(line);
    if (index >= fields.size()) return "";
    return fields[index];
}
static string getSchoolName(const string& schoolID) {
    string name = getField("schools.txt", schoolID, 1);
    return name.empty() ? "Unknown School" : name;
}
static bool schoolExists(const string& schoolID) {
    return idExists("schools.txt", schoolID);
}
static bool principalExists(const string& principalID) {
    return idExists("principals.txt", principalID);
}
static bool teacherExists(const string& teacherID) {
    return idExists("teachers.txt", teacherID);
}
static bool studentExists(const string& studentID) {
    return idExists("students.txt", studentID);
}
static string getTeacherSchoolID(const string& teacherID) {
    return getField("teachers.txt", teacherID, 6);
```

```
}
static string getPrincipalSchoolID(const string& principalID) {
    return getField("principals.txt", principalID, 6);
}
static string getStudentSchoolID(const string& studentID) {
    return getField("students.txt", studentID, 6);
}
static string getStudentName(const string& studentID) {
    return getField("students.txt", studentID, 2);
}
static bool studentBelongsToSchool(const string& studentID, const string& schoolID) {
    return studentExists(studentID) && getStudentSchoolID(studentID) == schoolID;
}
static void showStudentProfile(const string& id) {
    string line;
    if (!findByID("students.txt", id, line)) {
        cout << "Student Not Found!\n";
        return;
    }
    vector<string> f = split(line);
    cout << "\nID: " << (f.size() > 0 ? f[0] : "") << '\n';
    cout << "Name: " << (f.size() > 2 ? f[2] : "") << '\n';
    cout << "DOB: " << (f.size() > 3 ? f[3] : "") << '\n';
    cout << "Phone: " << (f.size() > 5 ? f[5] : "") << '\n';
    cout << "School ID: " << (f.size() > 6 ? f[6] : "") << '\n';
    cout << "School: " << (f.size() > 6 ? getSchoolName(f[6]) : "") << '\n';
    cout << "Class: " << (f.size() > 7 ? f[7] : "") << '\n';
    cout << "Section: " << (f.size() > 8 ? f[8] : "") << '\n';
    cout << "Roll No: " << (f.size() > 9 ? f[9] : "") << '\n';
    cout << "Parent: " << (f.size() > 10 ? f[10] : "") << '\n';
}
```

```
static bool searchStudent(const string& id, const string& schoolID = "") {
    string line;
    if (!findByID("students.txt", id, line)) return false;
    vector<string> f = split(line);
    if (!schoolID.empty() && (f.size() <= 6 || f[6] != schoolID)) return false;
    cout << line << '\n';
    return true;
}
static bool searchTeacher(const string& id, const string& schoolID = "") {
    string line;
    if (!findByID("teachers.txt", id, line)) return false;
    vector<string> f = split(line);
    if (!schoolID.empty() && (f.size() <= 6 || f[6] != schoolID)) return false;
    cout << line << '\n';
    return true;
}
static bool updateResult(const string& studentID, const string& subject, int newMarks) {
    ifstream file("results.txt");
    ofstream temp("temp_sms.txt");
    if (!file.is_open() || !temp.is_open()) return false;
    bool updated = false;
    string line;
    while (getline(file, line)) {
        vector<string> f = split(line);
        if (f.size() >= 3 && f[0] == studentID && f[1] == subject) {
            temp << studentID << '|' << subject << '|' << newMarks << '\n';
            updated = true;
        } else {
            temp << line << '\n';
        }
    }
```

```
        file.close();
        temp.close();
        remove("results.txt");
        rename("temp_sms.txt", "results.txt");
        return updated;
    }
    static bool deleteSchool(const string& schoolID) {
        return removeSchoolByID(schoolID);
    }
};
class User {
protected:
    string id;
    string password;
    string name;
    string dob;
    string aadhar;
    string phone;
public:
    User() = default;
    void setBasicInfo(const string& id, const string& password, const string& name,
                      const string& dob, const string& aadhar, const string& phone) {
        this->id = id;
        this->password = password;
        this->name = name;
        this->dob = dob;
        this->aadhar = aadhar;
        this->phone = phone;
    }
    string getID() const { return id; }
    string getPassword() const { return password; }
```

```
    string getName() const { return name; }
    string getDOB() const { return dob; }
    string getAadhar() const { return aadhar; }
    string getPhone() const { return phone; }
    virtual void display() const {
        cout << "\nID: " << id << '\n';
        cout << "Name: " << name << '\n';
        cout << "DOB: " << dob << '\n';
        cout << "Phone: " << phone << '\n';
    }
    virtual ~User() = default;
};
class Parent : public User {
    string studentID;
    string studentName;
public:
    bool login(const string& loginID, const string& pass) {
        return FileManager::verifyParentLogin(loginID, pass);
    }
    void displayMenu() {
        while (true) {
            cls();
            cout << "===== PARENT PANEL =====\n\n";
            cout << "1. View Student Profile\n";
            cout << "2. View Attendance\n";
            cout << "3. View Results\n";
            cout << "4. View Fees\n";
            cout << "5. View Notifications\n";
            cout << "6. Submit Complaint\n";
            cout << "7. Leave School Request\n";
            cout << "8. Logout\n";
```

```
        int choice;
          if (!menuChoice("\nEnter Choice: ", choice, 1, 8)) return;
          switch (choice) {
             case 1: viewStudentProfile(); break;
             case 2: viewAttendance(); break;
             case 3: viewResults(); break;
             case 4: viewFees(); break;
             case 5: viewNotifications(); break;
             case 6: submitComplaint(); break;
             case 7: leaveSchoolRequest(); break;
             case 8: return;
          }
        }
    }
private:
    void viewStudentProfile() {
        cls();
        string id;
        if (!inputText("Enter Student ID (or back): ", id)) return;
        cout << "\n===== STUDENT PROFILE =====\n";
        FileManager::showStudentProfile(id);
        pauseScreen();
    }
    void viewAttendance() {
        cls();
        cout << "===== ATTENDANCE =====\n\n";
        FileManager::loadAttendance();
        pauseScreen();
    }
    void viewResults() {
        cls();
```

```
    cout << "===== RESULTS =====\n\n";
    FileManager::loadResults();
    pauseScreen();
}
void viewFees() {
    cls();
    cout << "===== FEES =====\n\n";
    FileManager::loadFees();
    pauseScreen();
}
void viewNotifications() {
    cls();
    cout << "===== NOTIFICATIONS =====\n\n";
    FileManager::loadNotifications();
    pauseScreen();
}
void submitComplaint() {
    cls();
    string parentID, complaint;
    if (!inputText("Enter Parent ID (or back): ", parentID)) return;
    if (!inputText("Enter Complaint (or back): ", complaint)) return;

    if (!FileManager::saveComplaint(parentID + "|" + complaint)) {
        cout << "Unable to save complaint.\n";
    } else {
        cout << "\nComplaint Submitted Successfully!\n";
    }
    pauseScreen();
}
void leaveSchoolRequest() {
    cls();
```

```
        string studentID, reason;
        if (!inputText("Enter Student ID (or back): ", studentID)) return;
        if (!inputText("Enter Reason (or back): ", reason)) return;

        if (!FileManager::saveLeaveRequest(studentID + "|" + reason)) {
            cout << "Unable to save request.\n";
        } else {
            cout << "\nLeave Request Sent!\n";
        }
        pauseScreen();
    }
};
class Student : public User {
public:
    bool login(const string& loginID, const string& pass) {
        return FileManager::verifyStudentLogin(loginID, pass);
    }
    void displayMenu() {
        while (true) {
            cls();
            cout << "===== STUDENT PANEL =====\n\n";
            cout << "1. View Profile\n";
            cout << "2. View Attendance\n";
            cout << "3. View Results\n";
            cout << "4. View Fees\n";
            cout << "5. View Notifications\n";
            cout << "6. Logout\n";
            int choice;
            if (!menuChoice("\nEnter Choice: ", choice, 1, 6)) return;
            switch (choice) {
                case 1: viewProfile(); break;
```

```
            case 2: viewAttendance(); break;
            case 3: viewResults(); break;
            case 4: viewFees(); break;
            case 5: viewNotifications(); break;
            case 6: return;
          }
        }
    }
private:
    void viewProfile() {
        cls();
        string id;
        if (!inputText("Enter Student ID (or back): ", id)) return;
        cout << "\n===== PROFILE =====\n";
        FileManager::showStudentProfile(id);
        pauseScreen();
    }
    void viewAttendance() {
        cls();
        cout << "===== ATTENDANCE =====\n\n";
        FileManager::loadAttendance();
        pauseScreen();
    }
    void viewResults() {
        cls();
        cout << "===== RESULTS =====\n\n";
        FileManager::loadResults();
        pauseScreen();
    }
    void viewFees() {
        cls();
```

```
        cout << "===== FEES =====\n\n";
        FileManager::loadFees();
        pauseScreen();
    }
    void viewNotifications() {
        cls();
        cout << "===== NOTIFICATIONS =====\n\n";
        FileManager::loadNotifications();
        pauseScreen();
    }
};
class Teacher : public User {
    string schoolID;
    string subject;
public:
    bool login(const string& loginID, const string& pass) {
    if (!FileManager::verifyTeacherLogin(loginID, pass))
        return false;
    id = loginID;
    schoolID = FileManager::getTeacherSchoolID(loginID);
    return true;
}
    void displayMenu() {
        schoolID = FileManager::getTeacherSchoolID(id);
        while (true) {
            cls();
            cout << "===== TEACHER PANEL =====\n";
            cout << "School ID: " << schoolID << " (" << FileManager::getSchoolName(schoolID) <<
")\n\n";
            cout << "1. Admit Student\n";
            cout << "2. View Students\n";
```

```
            cout << "3. Mark Attendance\n";
            cout << "4. Enter Result\n";
            cout << "5. Set Fees\n";
            cout << "6. Search Student\n";
            cout << "7. Update Result\n";
            cout << "8. Request Student Removal\n";
            cout << "9. Send Notification\n";
            cout << "10. Logout\n";
            int choice;
            if (!menuChoice("\nEnter Choice: ", choice, 1, 10)) return;
            switch (choice) {
                case 1: admitStudent(); break;
                case 2: viewStudents(); break;
                case 3: markAttendance(); break;
                case 4: enterResult(); break;
                case 5: setFees(); break;
                case 6: searchStudent(); break;
                case 7: updateResult(); break;
                case 8: requestStudentRemoval(); break;
                case 9: sendNotification(); break;
                case 10: return;
            }
        }
    }
private:
    void admitStudent() {
        cls();
        cout << "===== ADMIT STUDENT =====\n";
        cout << "School: " << FileManager::getSchoolName(schoolID)
             << " [" << schoolID << "]\n\n";
        string studentID = FileManager::nextID("students.txt", "ST");
```

```
string password, name, dob, aadhar, phone, parentName;
int classNo, rollNo;
char section;
cout << "Generated Student ID: " << studentID << "\n\n";
if (!inputText("Enter Password (or back): ", password)) return;
if (!inputText("Enter Student Name (or back): ", name)) return;
if (!inputText("Enter DOB (or back): ", dob)) return;
if (!inputText("Enter Aadhar (or back): ", aadhar)) return;
if (!inputText("Enter Phone (or back): ", phone)) return;
if (!inputInt("Enter Class (or back): ", classNo, 1, 12)) return;
if (!inputChar("Enter Section (A/B/C... or back): ", section)) return;
section = static_cast<char>(toupper(static_cast<unsigned char>(section)));
if (!inputInt("Enter Roll Number (or back): ", rollNo, 1)) return;
if (!inputText("Enter Parent Name (or back): ", parentName)) return;
string studentData = studentID + "|" + password + "|" + name + "|" + dob + "|" +
                     aadhar + "|" + phone + "|" + schoolID + "|" +
                     to_string(classNo) + "|" + string(1, section) + "|" +
                     to_string(rollNo) + "|" + parentName;
if (!FileManager::saveStudent(studentData)) {
   cout << "\nUnable to save student.\n";
   pauseScreen();
   return;
}
string parentID = FileManager::nextID("parents.txt", "PA");
string parentData = parentID + "|" + password + "|" + parentName + "|" + studentID;
FileManager::saveParent(parentData);
cout << "\nStudent Admitted Successfully!\n";
cout << "Student ID : " << studentID << '\n';
cout << "Parent ID  : " << parentID << '\n';
cout << "School ID  : " << schoolID << '\n';
pauseScreen();
```

```
    }
    void viewStudents() {
        cls();
        cout << "===== STUDENTS OF " << FileManager::getSchoolName(schoolID) << "
=====\n\n";
        vector<string> students = FileManager::readAll("students.txt");
        bool found = false;
        for (const string& line : students) {
            vector<string> f = FileManager::split(line);
            if (f.size() > 6 && f[6] == schoolID) {
                cout << line << '\n';
                found = true;
            }
        }
        if (!found) cout << "No students found.\n";
        pauseScreen();
    }
    bool validateStudentForThisSchool(const string& studentID) {
        if (!FileManager::studentExists(studentID)) {
            cout << "Student Not Found!\n";
            return false;
        }
        if (!FileManager::studentBelongsToSchool(studentID, schoolID)) {
            cout << "Invalid student: this student does not belong to your school.\n";
            return false;
        }
        return true;
    }
    void markAttendance() {
        cls();
        string studentID, date;
```

```
    char status;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputText("Enter Date (DD-MM-YYYY or back): ", date)) return;
    if (!inputChar("Enter Status (P/A or back): ", status)) return;
    status = static_cast<char>(toupper(static_cast<unsigned char>(status)));
    if (status != 'P' && status != 'A') {
        cout << "Invalid option\n";
        pauseScreen();
        return;
    }
    if (FileManager::saveAttendance(studentID + "|" + date + "|" + string(1, status)))
        cout << "\nAttendance Marked Successfully!\n";
    else cout << "Unable to save attendance.\n";
    pauseScreen();
}
void enterResult() {
    cls();
    string studentID, subject;
    int marks;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputText("Enter Subject (or back): ", subject)) return;
    if (!inputInt("Enter Marks (0-100 or back): ", marks, 0, 100)) return;
    if (FileManager::saveResult(studentID + "|" + subject + "|" + to_string(marks)))
        cout << "\nResult Saved Successfully!\n";
    else cout << "Unable to save result.\n";
    pauseScreen();
}
void setFees() {
    cls();
```

```
    string studentID, status;
    int amount;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputInt("Enter Fees Amount (or back): ", amount, 0)) return;
    if (!inputText("Enter Status (Paid/Unpaid or back): ", status)) return;
    if (FileManager::saveFees(studentID + "|" + to_string(amount) + "|" + status))
        cout << "\nFees Saved Successfully!\n";
    else cout << "Unable to save fees.\n";
    pauseScreen();
}
void searchStudent() {
    cls();
    string id;
    if (!inputText("Enter Student ID (or back): ", id)) return;
    cout << "\n===== SEARCH RESULT =====\n";
    if (!FileManager::searchStudent(id, schoolID))
        cout << "Student Not Found!\n";
    pauseScreen();
}
void updateResult() {
    cls();
    string studentID, subject;
    int marks;
    if (!inputText("Enter Student ID (or back): ", studentID)) return;
    if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
    if (!inputText("Enter Subject (or back): ", subject)) return;
    if (!inputInt("Enter New Marks (0-100 or back): ", marks, 0, 100)) return;
    if (FileManager::updateResult(studentID, subject, marks))
        cout << "\nResult Updated Successfully!\n";
    else cout << "\nResult record not found!\n";
```

```
        pauseScreen();
    }
    void requestStudentRemoval() {
        cls();
        string studentID, reason;
        if (!inputText("Enter Student ID (or back): ", studentID)) return;
        if (!validateStudentForThisSchool(studentID)) { pauseScreen(); return; }
        if (!inputText("Enter Reason (or back): ", reason)) return;

        if (FileManager::saveRemovalRequest(studentID + "|" + reason))
            cout << "\nRemoval Request Sent Successfully!\n";
        else cout << "Unable to save request.\n";
        pauseScreen();
    }
    void sendNotification() {
        cls();
        string message;
        if (!inputText("Enter Notification (or back): ", message)) return;
        if (FileManager::saveNotification(id + "|" + schoolID + "|" + message))
            cout << "\nNotification Sent Successfully!\n";
        else cout << "Unable to save notification.\n";
        pauseScreen();
    }
};
class Principal : public User {
    string schoolID;
public:
    bool login(const string& loginID, const string& pass) {
        if (!FileManager::verifyPrincipalLogin(loginID, pass)) return false;
        id = loginID;
        schoolID = FileManager::getPrincipalSchoolID(loginID);
```

```
    return true;
  }
  void displayMenu() {
    while (true) {
      cls();
      cout << "===== PRINCIPAL PANEL =====\n";
      cout << "School ID: " << schoolID << " (" << FileManager::getSchoolName(schoolID) <<
")\n\n";
      cout << "1. Add Teacher\n";
      cout << "2. View Teachers\n";
      cout << "3. Search Teacher\n";
      cout << "4. Remove Teacher\n";
      cout << "5. View Students\n";
      cout << "6. View Attendance\n";
      cout << "7. View Results\n";
      cout << "8. View Complaints\n";
      cout << "9. View Removal Requests\n";
      cout << "10. Send Notification\n";
      cout << "11. Logout\n";
      int choice;
      if (!menuChoice("\nEnter Choice: ", choice, 1, 11)) return;
      switch (choice) {
        case 1: addTeacher(); break;
        case 2: viewTeachers(); break;
        case 3: searchTeacher(); break;
        case 4: removeTeacher(); break;
        case 5: viewStudents(); break;
        case 6: viewAttendance(); break;
        case 7: viewResults(); break;
        case 8: viewComplaints(); break;
        case 9: viewRemovalRequests(); break;
```

```
            case 10: sendNotification(); break;
            case 11: return;
        }
    }
  }
private:
  void addTeacher() {
    cls();
    cout << "===== ADD TEACHER =====\n";
    cout << "School: " << FileManager::getSchoolName(schoolID)
         << " [" << schoolID << "]\n\n";
    string teacherID = FileManager::nextID("teachers.txt", "TE");
    string password, name, dob, aadhar, phone, subject;
    cout << "Generated Teacher ID: " << teacherID << "\n\n";
    if (!inputText("Enter Password (or back): ", password)) return;
    if (!inputText("Enter Name (or back): ", name)) return;
    if (!inputText("Enter DOB (or back): ", dob)) return;
    if (!inputText("Enter Aadhar (or back): ", aadhar)) return;
    if (!inputText("Enter Phone (or back): ", phone)) return;
    if (!inputText("Enter Subject (or back): ", subject)) return;
    string data = teacherID + "|" + password + "|" + name + "|" + dob + "|" +
                  aadhar + "|" + phone + "|" + schoolID + "|" + subject;
    if (FileManager::saveTeacher(data)) {
      cout << "\nTeacher Added Successfully!\n";
      cout << "Teacher ID: " << teacherID << '\n';
      cout << "School ID : " << schoolID << '\n';
    } else {
      cout << "Unable to save teacher.\n";
    }
    pauseScreen();
  }
```

```
    void viewTeachers() {
        cls();
        cout << "===== TEACHERS OF " << FileManager::getSchoolName(schoolID) << "
=====\n\n";
        vector<string> teachers = FileManager::readAll("teachers.txt");
        bool found = false;
        for (const string& line : teachers) {
            vector<string> f = FileManager::split(line);
            if (f.size() > 6 && f[6] == schoolID) {
                cout << line << '\n';
                found = true;
            }
        }
        if (!found) cout << "No teachers found.\n";
        pauseScreen();
    }
    void searchTeacher() {
        cls();
        string id;
        if (!inputText("Enter Teacher ID (or back): ", id)) return;
        cout << "\n===== SEARCH RESULT =====\n";
        if (!FileManager::searchTeacher(id, schoolID))
            cout << "Teacher Not Found!\n";
        pauseScreen();
    }
    void removeTeacher() {
        cls();
        string teacherID;
        if (!inputText("Enter Teacher ID to Remove (or back): ", teacherID)) return;
        if (!FileManager::teacherExists(teacherID)) {
            cout << "Teacher Not Found!\n";
```

```
            pauseScreen();
            return;
        }
        if (FileManager::getTeacherSchoolID(teacherID) != schoolID) {
            cout << "You cannot remove a teacher from another school.\n";
            pauseScreen();
            return;
        }
        if (FileManager::removeByID("teachers.txt", teacherID))
            cout << "\nTeacher Removed Successfully!\n";
        else cout << "Unable to remove teacher.\n";
        pauseScreen();
    }
    void viewStudents() {
        cls();
        cout << "===== STUDENTS OF " << FileManager::getSchoolName(schoolID) << "
=====\n\n";
        vector<string> students = FileManager::readAll("students.txt");
        bool found = false;
        for (const string& line : students) {
            vector<string> f = FileManager::split(line);
            if (f.size() > 6 && f[6] == schoolID) {
                cout << line << '\n';
                found = true;
            }
        }
        if (!found) cout << "No students found.\n";
        pauseScreen();
    }
    void viewAttendance() {
        cls();
```

```
    cout << "===== ATTENDANCE RECORDS =====\n\n";
    FileManager::loadAttendance();
    pauseScreen();
}
void viewResults() {
    cls();
    cout << "===== RESULT RECORDS =====\n\n";
    FileManager::loadResults();
    pauseScreen();
}
void viewComplaints() {
    cls();
    cout << "===== COMPLAINTS =====\n\n";
    FileManager::loadComplaints();
    pauseScreen();
}
void viewRemovalRequests() {
    cls();
    cout << "===== REMOVAL REQUESTS =====\n\n";
    FileManager::loadRemovalRequests();
    pauseScreen();
}
void sendNotification() {
    cls();
    string message;
    if (!inputText("Enter Notification (or back): ", message)) return;
    if (FileManager::saveNotification(id + "|" + schoolID + "|" + message))
        cout << "\nNotification Sent Successfully!\n";
    else cout << "Unable to save notification.\n";
    pauseScreen();
}
```

```
};
class SuperAdmin {
   string password = "malik";
public:
   bool login(const string& enteredPassword) const {
      return enteredPassword == password;
   }
   void displayMenu() {
      while (true) {
         cls();
         cout << "===== SUPER ADMIN PANEL =====\n\n";
         cout << "1. Create School\n";
         cout << "2. View Schools\n";
         cout << "3. Add Principal\n";
         cout << "4. View Principals\n";
         cout << "5. Delete School\n";
         cout << "6. Remove Principal\n";
         cout << "7. Edit School\n";
         cout << "8. Global Notification\n";
         cout << "9. Logout\n";
         int choice;
         if (!menuChoice("\nEnter Choice: ", choice, 1, 9)) return;
         switch (choice) {
            case 1: createSchool(); break;
            case 2: viewSchools(); break;
            case 3: addPrincipal(); break;
            case 4: viewPrincipals(); break;
            case 5: deleteSchool(); break;
            case 6: removePrincipal(); break;
            case 7: editSchool(); break;
            case 8: globalNotification(); break;
```

```
            case 9: return;
        }
    }
  }
private:
  void createSchool() {
    cls();
    cout << "===== CREATE SCHOOL =====\n\n";
    string schoolName;
    int totalClasses, totalSections;
    string schoolID = FileManager::nextID("schools.txt", "SC");
    cout << "Generated School ID: " << schoolID << "\n\n";
    if (!inputText("Enter School Name (or back): ", schoolName)) return;
    if (!inputInt("Enter Total Classes (or back): ", totalClasses, 1)) return;
    if (!inputInt("Enter Total Sections (or back): ", totalSections, 1)) return;
    string data = schoolID + "|" + schoolName + "||" +
                  to_string(totalClasses) + "|" + to_string(totalSections);
    if (FileManager::saveSchool(data)) {
      cout << "\nSchool Created Successfully!\n";
      cout << "School ID: " << schoolID << '\n';
    } else {
      cout << "Unable to save school.\n";
    }
    pauseScreen();
  }
  void viewSchools() {
    cls();
    cout << "===== ALL SCHOOLS =====\n\n";
    FileManager::loadSchools();
    pauseScreen();
  }
```

```
    void addPrincipal() {
        cls();
        cout << "===== ADD PRINCIPAL =====\n\n";
        string schoolID;
        if (!inputText("Enter School ID (SC001) or back: ", schoolID)) return;
        if (!FileManager::schoolExists(schoolID)) {
            cout << "School ID not found! No random school names allowed\n";
            pauseScreen();
            return;
        }
        string principalID = FileManager::nextID("principals.txt", "PR");
        string password, name, dob, aadhar, phone;
        cout << "Generated Principal ID: " << principalID << "\n";
        cout << "School: " << FileManager::getSchoolName(schoolID) << " [" << schoolID <<
"]\n\n";
        if (!inputText("Enter Password (or back): ", password)) return;
        if (!inputText("Enter Name (or back): ", name)) return;
        if (!inputText("Enter DOB (or back): ", dob)) return;
        if (!inputText("Enter Aadhar (or back): ", aadhar)) return;
        if (!inputText("Enter Phone (or back): ", phone)) return;
        string data = principalID + "|" + password + "|" + name + "|" + dob + "|" +
                      aadhar + "|" + phone + "|" + schoolID;
        if (!FileManager::savePrincipal(data)) {
            cout << "Unable to save principal.\n";
            pauseScreen();
            return;
        }
        string schoolLine;
        if (FileManager::findByID("schools.txt", schoolID, schoolLine)) {
            vector<string> f = FileManager::split(schoolLine);
            if (f.size() >= 5) {
```

```
            f[2] = principalID;
            string updated = f[0];
            for (size_t i = 1; i < f.size(); ++i) updated += "|" + f[i];
            FileManager::updateLine("schools.txt", schoolID, updated);
        }
    }
    cout << "\nPrincipal Added Successfully!\n";
    cout << "Principal ID: " << principalID << '\n';
    cout << "School ID   : " << schoolID << '\n';
    pauseScreen();
}
void viewPrincipals() {
    cls();
    cout << "===== ALL PRINCIPALS =====\n\n";
    FileManager::loadPrincipals();
    pauseScreen();
}
void removePrincipal() {
    cls();
    string principalID;
    if (!inputText("Enter Principal ID to Remove (or back): ", principalID)) return;
    if (!FileManager::principalExists(principalID)) {
        cout << "Principal Not Found!\n";
        pauseScreen();
        return;
    }
    string schoolID = FileManager::getPrincipalSchoolID(principalID);
    if (FileManager::removeByID("principals.txt", principalID)) {
        string schoolLine;
        if (FileManager::findByID("schools.txt", schoolID, schoolLine)) {
            vector<string> f = FileManager::split(schoolLine);
```

```
            if (f.size() >= 5) {
                f[2] = "";
                string updated = f[0];
                for (size_t i = 1; i < f.size(); ++i) updated += "|" + f[i];
                FileManager::updateLine("schools.txt", schoolID, updated);
            }
        }
        cout << "\nPrincipal Removed Successfully!\n";
    } else {
        cout << "Unable to remove principal.\n";
    }
    pauseScreen();
}
void deleteSchool() {
    cls();
    string schoolID;
    if (!inputText("Enter School ID to Delete (or back): ", schoolID)) return;
    if (!FileManager::schoolExists(schoolID)) {
        cout << "School Not Found!\n";
        pauseScreen();
        return;
    }
    if (FileManager::deleteSchool(schoolID))
        cout << "\nSchool Deleted Successfully!\n";
    else
        cout << "Unable to delete school.\n";
    pauseScreen();
}
void editSchool() {
    cls();
    string schoolID, newName;
```

```
    if (!inputText("Enter School ID (or back): ", schoolID)) return;
    if (!FileManager::schoolExists(schoolID)) {
        cout << "School Not Found!\n";
        pauseScreen();
        return;
    }
    if (!inputText("Enter New School Name (or back): ", newName)) return;
    string line;
    if (!FileManager::findByID("schools.txt", schoolID, line)) {
        cout << "School Not Found!\n";
        pauseScreen();
        return;
    }
    vector<string> f = FileManager::split(line);
    if (f.size() < 5) {
        cout << "School record is damaged.\n";
        pauseScreen();
        return;
    }
    f[1] = newName;
    string updated = f[0];
    for (size_t i = 1; i < f.size(); ++i) updated += "|" + f[i];

    if (FileManager::updateLine("schools.txt", schoolID, updated))
        cout << "\nSchool Updated Successfully!\n";
    else
        cout << "Unable to update school.\n";
    pauseScreen();
}
void globalNotification() {
    cls();
```

```
        string message;
        if (!inputText("Enter Global Notification (or back): ", message)) return;
        if (FileManager::saveNotification("GLOBAL|" + message))
            cout << "\nGlobal Notification Sent!\n";
        else cout << "Unable to save notification.\n";
        pauseScreen();
    }
};
}
using namespace SMS;
int main() {
    while (true) {
        cls();
        cout << "==========================\n";
        cout << " SCHOOL MANAGEMENT SYSTEM\n";
        cout << "==========================\n\n";
        cout << "1. Principal\n";
        cout << "2. Teacher\n";
        cout << "3. Student\n";
        cout << "4. Parent\n\n";
        cout << "Type 'admin' for Super Admin\n";
        cout << "Type 'exit' to Exit\n";
        cout << "Type 'back' to stay here\n\n";
        cout << "Enter Choice: ";
        string choice;
        getline(cin >> ws, choice);
        string lower = lowerCopy(choice);
        if (lower == "exit") {
            cls();
            cout << "Thank You For Using SMS!\n";
            break;
```

```
    }
    if (lower == "back") continue;
    if (lower == "admin") {
        cls();
        SuperAdmin admin;
        string password;
        if (!inputText("Enter Admin Password (or back): ", password)) continue;
        if (admin.login(password)) {
            cout << "\nLogin Successful!\n";
            pauseScreen();
            admin.displayMenu();
        } else {
            cout << "\nWrong Password!\n";
            pauseScreen();
        }
    }else if (choice == "1") {
        cls();
        Principal p;
        string id, password;
        if (!inputText("Enter Principal ID (or back): ", id)) continue;
        if (!inputText("Enter Password (or back): ", password)) continue;
        if (p.login(id, password)) {
            cout << "\nLogin Successful!\n";
            pauseScreen();
            p.displayMenu();
        } else {
            cout << "\nInvalid Credentials!\n";
            pauseScreen();
        }
    }else if (choice == "2") {
        cls();
```

```
    Teacher t;
    string id, password;
    if (!inputText("Enter Teacher ID (or back): ", id)) continue;
    if (!inputText("Enter Password (or back): ", password)) continue;
    if (t.login(id, password)) {
        cout << "\nLogin Successful!\n";
        pauseScreen();
        t.displayMenu();
    } else {
        cout << "\nInvalid Credentials!\n";
        pauseScreen();
    }
}else if (choice == "3") {
    cls();
    Student s;
    string id, password;
    if (!inputText("Enter Student ID (or back): ", id)) continue;
    if (!inputText("Enter Password (or back): ", password)) continue;
    if (s.login(id, password)) {
        cout << "\nLogin Successful!\n";
        pauseScreen();
        s.displayMenu();
    } else {
        cout << "\nInvalid Credentials!\n";
        pauseScreen();
    }
}else if (choice == "4") {
    cls();
    Parent p;
    string id, password;
    if (!inputText("Enter Parent ID (or back): ", id)) continue;
```

```
            if (!inputText("Enter Password (or back): ", password)) continue;
            if (p.login(id, password)) {
                cout << "\nLogin Successful!\n";
                pauseScreen();
                p.displayMenu();
            } else {
                cout << "\nInvalid Credentials!\n";
                pauseScreen();
            }
        }else{
            invalidChoice();
        }
    }
    return 0;
}
```